

---

# Tracing the Roots: Trust Region Policy Optimization

---

**Anshu Arora**

Department of Electrical Engineering  
Indian Institute of Technology  
Mumbai, MH, 400076  
anshuarora2604@gmail.com

**Pranay Sharma**

Centre for Machine Intelligence and Data Science  
Indian Institute of Technology  
Mumbai, MH, 400076  
pranaysh@iitb.ac.in

## Abstract

This report examines the origins of the celebrated Trust Region Policy Optimization (TRPO) framework, an application of natural gradient methods in reinforcement learning. While the standard objective of an agent is to maximize cumulative reward, traditional policy gradient approaches are often highly sensitive to step-size selection because gradients are typically computed in the parameter space rather than the actual underlying policy space. TRPO addresses this instability by introducing a ‘trust region’ constraint based on the Kullback-Leibler divergence between different policies. This ensures that each policy update remains within a safe distance of the previous iteration. By leveraging the Fisher Information Matrix as an invariant metric to guide the natural gradient, the framework provides a robust optimization strategy that guarantees monotonic improvement of the policy.

## 1 Introduction

Reinforcement Learning (RL) provides a principled framework for learning decision-making strategies in an unsupervised manner, through interaction with an environment. Despite its strong theoretical foundations dating back to the mid-20th century Bellman [1954], Kaelbling et al. [1996], practical RL has historically been limited by computational challenges and instability in learning, especially in high-dimensional settings Dulac-Arnold et al. [2019].

A central goal in RL is to learn an optimal policy that maximizes cumulative reward in a Markov Decision Process (MDP). Classical approaches such as value iteration and policy iteration rely on dynamic programming principles and Bellman operators to iteratively improve policies. While these methods are theoretically sound, they often become infeasible in large or continuous state-action spaces, which is what more complex problems such as playing video games, or designing a controller for robotic arms require.

Policy gradient methods address this limitation by directly optimizing a parameterized policy using gradient-based techniques. These methods enable the use of popular and well-studied function approximators such as neural networks, making them suitable for complex environments. However, standard policy gradient approaches suffer from issues including sensitivity to parameterization, inefficient updates, and unstable learning dynamics.

To overcome these challenges, the notion of natural gradients was introduced, which accounts for the underlying geometry of the policy space by incorporating the Fisher information matrix. This leads to parameterization-invariant updates that better reflect meaningful changes in the policy distribution.

Building on these ideas, Trust Region Policy Optimization (TRPO) introduces a principled way to ensure stable and monotonic policy improvement. Instead of taking unconstrained gradient steps, TRPO constrains updates within a trust region defined by a divergence measure between successive policies. This results in improved performance and robustness in practice.

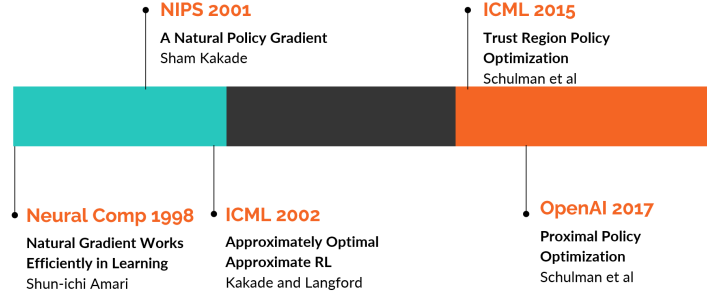


Figure 1: Timeline of advancements that led us towards TRPO

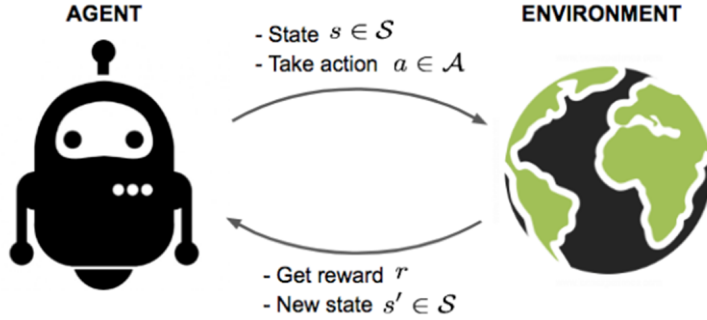


Figure 2: Basic framework of RL, reference: [lilianweng.github.io/posts/2018-02-19-rl-overview](https://lilianweng.github.io/posts/2018-02-19-rl-overview)

This report traces the conceptual and mathematical development leading to TRPO, starting from classical RL formulations, progressing through policy gradients and natural gradients, and culminating in the trust region framework for policy optimization, the timeline as shown in Fig. 1.

Let us begin our journey through time on this fascinating topic by first building up the required background in Section 2. We discuss the need for policy gradients in Section 3, and introduce natural gradients in Section 4. These ideas are built upon in Sections 5 and 6. We briefly discuss an important advancement of TRPO in 7, and conclude in 8.

## 2 Preliminaries

We formalize the RL setup and introduce the key quantities used throughout this report and existing literature. Fig. 2 provides a succinct representation of the RL framework, where the agent interacts directly with the environment.

**Markov Decision Process (MDP)** An RL problem is typically modeled as an MDP, which is characterized by the tuple  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ .

Here  $\mathcal{S}$  is the state space: the set of all possible configurations the agent can exist in and  $\mathcal{A}$  is the action space: the set of all allowable actions.  $P(s' | s, a)$  denotes the transition probability, that is in state  $s$ , if the agent takes an action  $a$ , its probability to go to the state  $s'$ ,  $r(s, a)$  is the reward function capturing the the gain upon choosing a particular action in a particular state. Note that the reward does not depend on the state we transition to, but only on our "intent".  $\gamma \in (0, 1)$  is the discount factor, which can be thought of as weighing rewards in the present more highly than the same reward in the future. A simple example is that one ice cream today has the same value to me as  $\frac{1}{\gamma}$  ice creams tomorrow.

In a nutshell, at each time step  $t$ , the agent observes a state  $s_t$ , takes an action  $a_t \sim \pi(\cdot | s_t)$  according to a policy  $\pi$ , which is a probability distribution over all states (q2) and receives a reward  $r(s_t, a_t)$ .

**Objective** The goal in infinite-time horizon RL is to find a policy  $\pi$  that maximizes the appropriate objective, as per the problem setting

*Discounted Return Objective* In the infinite-horizon setting with discounting, the goal is to find a policy  $\pi$  that maximizes:

$$J(\pi) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (1)$$

and the expectation is taken over trajectories generated by following policy  $\pi$ . Note, that since the rewards are upper bounded, introduction of the discount factor  $\gamma$  ensures that the infinite sum converges.

*Average Reward Objective* In the average reward setting, the objective is defined as to maximize the average reward per step:

$$\eta(\pi) = \sum_{s \in \mathcal{S}} \rho^{\pi}(s) \sum_{a \in \mathcal{A}} \pi(a | s) r(s, a), \quad (2)$$

where  $\rho^{\pi}(s)$  denotes the stationary state distribution under policy  $\pi$  (q3,  $\pi = \pi \rho^{\pi}(s)$ ), the guarantee of existence of which comes from assuming the underlying Markov Chain to be aperiodic and irreducible.

**Value Functions and Advantage** Now that we have the objectives defined, we move to evaluate the performance of a policy  $\pi$ , for which we define the following functions.

The *state-value function* represents the expected return starting from state  $s$  and following policy  $\pi$ :

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]. \quad (3)$$

The *action-value function* represents the expected return when taking action  $a$  in state  $s$  (a one-shot deviation) and thereafter following policy  $\pi$ :

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right]. \quad (4)$$

Note that these expectations are replaced with samples, and finding these comes under the term - policy evaluation, which has a rich class of algorithms (temporal difference, monte carlo methods) for finding the same (q4). The *advantage function* quantifies how much better an action  $a$  is compared to the policy's average behavior at state  $s$ :

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \quad (5)$$

**Bellman Equations** The value functions satisfy recursive relationships known as the Bellman equations.

For a fixed policy  $\pi$ , the state-value function satisfies:

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^{\pi}(s')]]. \quad (6)$$

The optimal value function satisfies the Bellman optimality equation:

$$V^*(s) = \max_{a \in \mathcal{A}} [r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^*(s')]]. \quad (7)$$

**Classical Methods** Two fundamental approaches for computing optimal policies are:

*Value Iteration:* repeatedly apply the Bellman optimality operator to converge to  $V^*$ , and then extract the optimal policy. This guarantees convergence (at a geometric rate), given standard assumptions.

*Policy Iteration:* alternate between policy evaluation (computing  $V^{\pi}$ ) and policy improvement (updating  $\pi$  greedily with respect to  $Q^{\pi}$ ). This procedure is guaranteed to converge to the optimal policy in a finite number of iterations, although the worst-case complexity can be exponential in the size of the state space.

While these methods provide strong theoretical guarantees, they become computationally expensive or intractable in large-scale or continuous domains, motivating the need for policy gradient methods discussed in the next section.

### 3 Policy Gradient Methods

Classical dynamic programming approaches, such as value iteration and policy iteration become computationally prohibitive in high-dimensional or continuous domains, where storing and updating value functions is intractable Kakade [2001].

Policy gradient methods address these challenges by directly optimizing a parameterized policy, avoiding the need for explicit value function computation over the entire state space. This makes them particularly suitable for large-scale problems and enables the use of function approximators such as neural networks.

Let  $\pi_\theta(a | s)$  denote a stochastic policy parameterized by  $\theta \in \mathbb{R}^d$ . Notice that this may be mildly restrictive; for example, softmax parametrization would not allow the probability for certain actions to be zero. However, having some kind of parameterization or function approximation is a standard practice in RL (eg. Linear Function Approximation methods, Tsitsiklis and Van Roy [1996])(q7).

The objective is to maximize the expected discounted return, as given in equation (1)

A key result, known as the policy gradient theorem, provides an expression for the gradient of the objective through as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) A^{\pi_\theta}(s, a)]. \quad (8)$$

Using stochastic estimates of this gradient, the parameters are updated via gradient ascent:

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_k), \quad (9)$$

where  $\alpha$  is the step size.

**Limitations of Standard Policy Gradients** Despite their flexibility, standard policy gradient methods suffer from several issues:

- *Parameterization Dependence:* The gradient is computed in parameter space, and different parameterizations of the same policy can lead to different update directions and trajectories.
- *Inefficient Updates:* Small changes in parameters may result in large changes in the policy distribution, or vice versa.
- *Plateau Behavior:* Optimization can stagnate away from the optimal policy due to poorly scaled gradients.

These limitations arise because the Euclidean gradient does not account for the geometry of the policy space, which lies in a simplex. This motivates the introduction of natural gradients, which incorporate a notion of distance between probability distributions, and were introduced as a tool for efficient learning in Amari [1998].

### 4 Natural Policy Gradient

We now consider policy optimization in the average reward setting. The objective is defined as the long-term average reward, as given in equation (2).

In this setting, the action-value function can be expressed as the differential return:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} (r_t - \eta(\pi)) \mid s_0 = s, a_0 = a \right]. \quad (10)$$

Under standard assumptions (e.g., the Markov chain induced by  $\pi$  is ergodic), this infinite sum is well-defined. Intuitively, since  $\eta(\pi)$  represents the steady-state average reward, the centered rewards  $(r_t - \eta(\pi))$  have zero mean in the long run, and their cumulative sum remains bounded in expectation (q13).

The gradient of the objective takes the form:

$$\nabla_\theta \eta(\theta) = \sum_s \rho^\pi(s) \sum_a \nabla_\theta \pi_\theta(a | s) Q^\pi(s, a). \quad (11)$$

**Geometry of Policy Space** A key observation is that a policy defines a probability distribution over actions. Therefore, meaningful changes in the policy should be measured in the space of distributions rather than in parameter space. This motivates equipping the parameter space with a geometry that reflects changes in the policy.

This geometry is captured by the Fisher information matrix. For a fixed state  $s$ , define:

$$F_s(\theta) = \mathbb{E}_{a \sim \pi_\theta(\cdot | s)} [\nabla_\theta \log \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s)^\top]. \quad (12)$$

Aggregating over states, so that we bring back the dependance on the underlying MDP, gives:

$$F(\theta) = \mathbb{E}_{s \sim \rho^\pi} [F_s(\theta)]. \quad (13)$$

The Fisher matrix defines a notion of distance in policy space and is invariant to reparameterization, ensuring that updates depend only on the induced distribution and not on how it is parameterized (q15).

**Natural Gradient Update** The natural gradient is obtained by preconditioning the standard gradient with the inverse Fisher matrix:

$$\tilde{\nabla}_\theta \eta(\theta) = F(\theta)^{-1} \nabla_\theta \eta(\theta). \quad (14)$$

This leads to the update:

$$\theta_{k+1} = \theta_k + \alpha F(\theta_k)^{-1} \nabla_\theta \eta(\theta_k), \quad (15)$$

which corresponds to the direction of steepest ascent measured in the space of probability distributions, leading to benefits in terms of convergence.

**Connection to Policy Improvement** Natural gradients can be interpreted as moving in the direction of greatest performance improvement for a fixed change in the policy distribution. In particular, for policies parameterized in the exponential family,

$$\pi_\theta(a | s) \propto \exp(\theta^\top \phi(s, a)), \quad (16)$$

it can be shown that taking a natural gradient step with an infinitely large step size recovers greedy policy improvement Kakade and Langford [2002]:

$$\pi_{\text{greedy}}(s) = \arg \max_a Q^\pi(s, a). \quad (17)$$

Thus, natural policy gradient can be viewed as interpolating between small, stable updates and full greedy policy improvement, and this is its selling point (q5).

**Limitations** While natural gradients remove sensitivity to parameterization, they do not explicitly control how much the policy changes in a single update. The same step size can lead to very different changes in the policy distribution depending on the local curvature. Moreover, updates are still computed in parameter space, with the geometry only implicitly accounted for through the Fisher matrix.

This motivates the need for methods with that directly change the policy, instead of operating on the parameter space.

## 5 Conservative Policy Iteration

Conservative Policy Iteration (CPI) way to ensure monotonic policy improvement by constructing a new policy as a mixture of the current policy  $\pi$  and an improved policy  $\pi'$ :

$$\pi_{\text{new}} = (1 - \alpha)\pi + \alpha\pi', \quad (18)$$

where  $\alpha \in (0, 1]$  controls the step size.

Although CPI provides strong theoretical guarantees, it relies on mixture policies, which are often impractical to represent and optimize, especially when policies are parameterized by neural networks. Additionally, selecting an appropriate mixing coefficient  $\alpha$  can be challenging in practice.

The key insight leading to TRPO is that instead of forming mixture policies, one can directly constrain how much the new policy deviates from the old policy. This perspective allows us to retain the theoretical guarantees of CPI while enabling efficient optimization over general parameterized policies. In the next section, we formalize this idea and derive the TRPO objective.

## 6 Trust Region Policy Optimization

TRPO Schulman et al. [2015] extends policy improvement guarantees to general stochastic policies. To reiterate, instead of constructing mixture policies as in Conservative Policy Iteration, TRPO constrains how much the policy is allowed to change at each update. The new policy is restricted to lie within a safe *trust region*, ensuring stable improvement.

**Framework** The objective is given in equation (2). Equivalently, this can be written as:

$$J(\pi) = \sum_s \rho^\pi(s) \sum_a \pi(a | s) R(s, a). \quad (19)$$

The goal is to solve:

$$\max_{\pi} J(\pi). \quad (20)$$

Using the performance difference identity Kakade and Langford [2002], we can write:

$$J(\tilde{\pi}) = J(\pi) + \sum_s \rho^{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a | s) A^\pi(s, a). \quad (21)$$

This is difficult to solve in practice, since calculating the stationary distribution corresponding to  $\tilde{\pi}$  is computationally intensive, and for maximizing, we may have to sweep over several such policies. This motivates the surrogate objective:

$$L_\pi(\tilde{\pi}) = J(\pi) + \sum_s \rho^\pi(s) \sum_a \tilde{\pi}(a | s) A^\pi(s, a), \quad (22)$$

where we have replaced  $\rho^{\tilde{\pi}}$  with the previous stationary distribution  $\rho^\pi$ . The surrogate optimization problem is then:

$$\max_{\tilde{\pi}} L_\pi(\tilde{\pi}). \quad (23)$$

Note that the surrogate objective and true objective are related as:

$$J(\pi_{\text{new}}) \leq L_\pi(\pi_{\text{new}}) + \frac{2\epsilon\gamma}{(1-\gamma)^2} \alpha^2, \quad (24)$$

where  $\pi_{\text{new}}$  comes from the optimization in equation (23), and

$$\epsilon = \max_s \left| \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot | s)} [A^\pi(s, a)] \right|. \quad (25)$$

To remove dependence on the mixture coefficient  $\alpha$ , we relate it to a divergence between policies. Using total variation distance, we define:

$$D_{\text{TV}}^{\max}(\pi, \tilde{\pi}) = \max_s D_{\text{TV}}(\pi(\cdot | s) \| \tilde{\pi}(\cdot | s)), \quad (26)$$

and using the well-known inequality

$$D_{\text{TV}}(p \| q)^2 \leq D_{\text{KL}}(p \| q), \quad (27)$$

We then use the maximum KL divergence over all states,

$$D_{\text{KL}}^{\max}(\pi, \tilde{\pi}) = \max_s D_{\text{KL}}(\pi(\cdot | s) \| \tilde{\pi}(\cdot | s)). \quad (28)$$

This leads to:

$$J(\tilde{\pi}) \leq L_\pi(\tilde{\pi}) + C D_{\text{KL}}^{\max}(\pi, \tilde{\pi}), \quad (29)$$

where

$$C = \frac{2\epsilon\gamma}{(1-\gamma)^2}. \quad (30)$$

Notice that we can now reformulate the optimization problem. Instead of penalizing the KL divergence through the constant  $C$ , we can use an alternate constraint  $\delta$ . This is done because the value of  $C$  obtained from equation (30) results in very slow updates, and we can choose our  $\delta$  as we like. We elaborate more on this shortly.

The beauty of TRPO is that it replaces the need for convexity with a monotonic improvement guarantee. Even in a jagged, non-convex landscape, TRPO ensures that every step the agent takes is a "safe" step that won't crash the performance (q12).

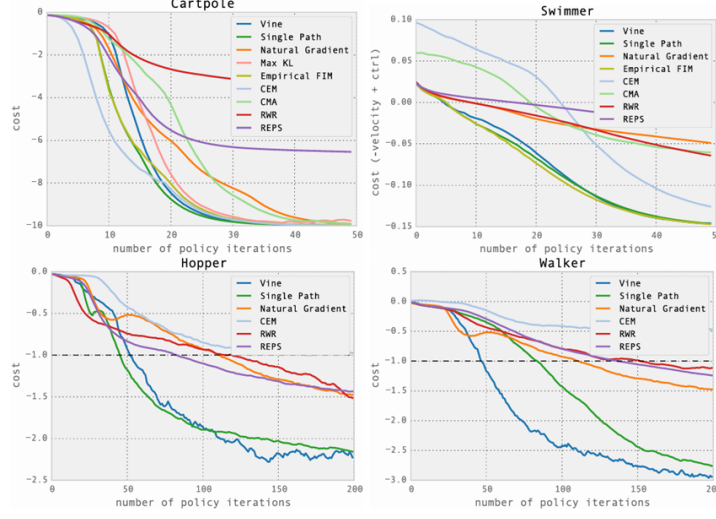


Figure 3: Two variants of TRPO algorithms, vine and single path performing on different Atari games. In the four games, they lead to rapid reduction in cost with fewer number of iterations.

**Practical Optimization Problem** We approximate the max-KL divergence using an average KL:

$$\bar{D}_{\text{KL}}(\theta_1, \theta_2) := \mathbb{E}_{s \sim \rho} [D_{\text{KL}}(\pi_{\theta_1}(\cdot | s) \| \pi_{\theta_2}(\cdot | s))] . \quad (31)$$

By replacing this, and changing the penalization in the objective, to the constraint, our final optimization problem becomes:

$$\min_{\tilde{\theta}} L_{\theta}(\tilde{\theta}) \quad (32)$$

$$\text{subject to } \bar{D}_{\text{KL}}(\theta, \tilde{\theta}) \leq \delta. \quad (33)$$

In practice, expectations are replaced with sample averages. The surrogate objective can be written as:

$$\mathbb{E}_{s \sim \rho_{\theta}, a \sim \pi_{\theta}} \left[ \frac{\pi_{\tilde{\theta}}(a | s)}{\pi_{\theta}(a | s)} Q_{\theta}(s, a) \right], \quad (34)$$

subject to:

$$\mathbb{E}_{s \sim \rho_{\theta}} [D_{\text{KL}}(\pi_{\theta}(\cdot | s) \| \pi_{\tilde{\theta}}(\cdot | s))] \leq \delta. \quad (35)$$

Notice the role of  $\delta$ . When it is large, we are open to having a larger trust region constraint, meaning we can afford to have bigger changes in the policy, and when it is small we take more cautious improvement steps (q18), which leads to slow, but safer convergence.

We now see the results of TRPO, benchmarked against different RL algorithms in Fig. 3. "Vine" and "Single Path" are different methods for replacing the expectation with sample averages, as given in (34)

It is important to note that this speed of convergence comes with a tradeoff, TRPO is computationally intensive, and in some simple settings it can be overkill. Thus it is the choice of the engineer to weigh the pros and cons of different methods! (q17)

To summarize, TRPO performs policy optimization by maximizing a surrogate objective while constraining policy updates within a trust region defined by KL divergence. This ensures stable, monotonic improvement, but requires computational resources.

## 7 Towards PPO

While TRPO provides a principled approach to stable policy improvement, its practical implementation is computationally intensive and algorithmically complex. In particular, it requires second-order

optimization (via Fisher matrix approximations), conjugate gradient methods, and careful line search to enforce the trust region constraint.

Proximal Policy Optimization (PPO) Schulman et al. [2017] was introduced to retain the core idea of controlled policy updates while significantly simplifying the optimization procedure. Instead of enforcing a hard constraint on the KL divergence, PPO incorporates this notion directly into the objective function.

The central idea is to prevent large policy updates by restricting how much the new policy can deviate from the old one. This is typically achieved using a clipped objective:

$$L(\theta) = \mathbb{E} [\min (r(\theta)A, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon)A)], \quad (36)$$

where  $r(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$  is the probability ratio.

This formulation avoids the need for second-order methods and explicit constraints, enabling the use of standard first-order optimization.

In essence, PPO can be viewed as a first-order approximation to TRPO: it preserves the idea of limiting policy updates (a "soft" trust region), while offering a simpler, and more scalable algorithm.

All these foundational results, that happened in the last decade have powered so much of today's AI. We see PPO algorithms used in fine-tuning of LLMs, and there are several popular papers that are building upon these results. It is fascinating to me how, quite literally, researchers "stand on the shoulders of giants" and continue pushing the frontiers of innovation.

## 8 Conclusion

This report traced major landmarks in policy optimization methods in reinforcement learning, culminating in Trust Region Policy Optimization (TRPO). Starting from the classical Markov Decision Process framework, we reviewed value-based methods and highlighted their limitations in high-dimensional settings. This motivated the shift toward policy gradient methods, which directly optimize parameterized policies but suffer from instability and sensitivity to parameterization.

The introduction of natural policy gradients addressed these issues by incorporating the geometry of the policy space through the Fisher information matrix, leading to parameterization-invariant updates. Building on this, Conservative Policy Iteration (CPI) provided theoretical guarantees for monotonic policy improvement, albeit with practical limitations due to its reliance on mixture policies.

TRPO unified these ideas by introducing a trust region constraint based on KL divergence, enabling stable and efficient policy updates for general parameterized policies. Its formulation connects naturally to the natural gradient while ensuring controlled changes in the policy distribution, resulting in strong empirical performance.

However, the computational and implementation complexity of TRPO motivated the development of Proximal Policy Optimization (PPO), which simplifies the optimization process.

Overall, the progression from standard policy gradients to TRPO and PPO illustrates a central theme in reinforcement learning: the importance of aligning optimization procedures with the underlying structure of the problem, particularly the geometry of probability distributions. This perspective continues to inform modern advancements in scalable and stable policy optimization methods.

## References

- Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- Richard Bellman. The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6):503–515, 1954.
- Gabriel Dulac-Arnold, Daniel Mankowitz, and Todd Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.
- Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.

- Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the nineteenth international conference on machine learning*, pages 267–274, 2002.
- Sham M Kakade. A natural policy gradient. *Advances in neural information processing systems*, 14, 2001.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- John Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. *Advances in neural information processing systems*, 9, 1996.